



University of Piraeus

School of Information and Communication Technologies

Department of Digital Systems

Level: Master's Programme of Studies

Advanced Topics in Cybersecurity and Artificial Intelligence

LLM-Enhanced SSH Honeypot and Attacker Interaction Analytics

Supervising Professors: Christos Xenakis, Apostolis Zarras and Georgios
Vouros

Ioannis Kalaitzidis

ioannis_kalaitzidis@ssl-unipi.gr

Piraeus

Introduction

Honeypots are one of the most instructive defence techniques in cybersecurity. Instead of trying to block the attacker, they allow them to enter a controlled environment and then study every action they take. This transforms a successful intrusion into intelligence.

The subject of this assignment is the construction of an SSH honeypot enriched with content generated by Large Language Models (LLMs), combined with a complete analytics pipeline for attacker session analysis.

A fundamental design principle throughout the entire assignment is that the analysis must be based on real events rather than speculation. Every number in this report comes from actual JSON logs produced by the system's execution — not from estimates or manually crafted data.

The assignment answers seven tasks: design the honeypot and threat model (Task 1), generate decoy artefacts with LLMs (Task 2), record and reconstruct sessions (Task 3), run controlled attacker playbooks (Task 4), analyse behaviour and intent (Task 5), evaluate deception quality (Task 6), and address ethics, safety and operational use (Task 7).

Implementation structure

The system is not a single program but a chain of discrete Python scripts, each with one clear responsibility.

The entire pipeline is reproducible via a fixed random seed (value 25012), so every number in this report comes from the same, deterministic execution.

Task 1: Honeypot Design and Development

The assignment does not specify a particular scenario, but asks for a convincing environment with a consistent backstory. The chosen scenario was a deployment server for a medium-sized company's web application.

In contrast to a simple banner mimicry, the honeypot implements real SSH protocol via the paramiko library — meaning actual handshake and authentication — and then presents a fully simulated shell environment.

Threat Model

The honeypot is designed to study an attacker who has already gained shell access (post-exploitation stage). The primary threat actor is a semi-automated attacker conducting reconnaissance and attempting lateral movement.

The term lateral movement describes the attacker's attempt to move from the machine they initially compromised to other machines inside the same network.

Architecture

The architecture is organised into four layers, each with a distinct role.

1. **The network layer (paramiko Transport):** Accepts the incoming TCP connection, starts the SSH dialogue as server, and presents a realistic server key.
2. **The authentication layer:** The honeypot accepts connection attempts (typical honeypot behaviour, so the attacker proceeds to the shell rather than giving up).
3. **The shell emulator:** This is the most complex layer. It receives each command line, parses it, produces a realistic response (either from a pre-defined dictionary or LLM-generated content), and simultaneously records every interaction in the logs.
4. **The Virtual File System (VFS):** Presents the attacker with an apparently complete file system, combining two sources: the decoy artefacts and a set of synthetic files providing realistic directory structure.

Files implementing Task 1: 02_ssh_honeypot.py (honeypot, shell emulator, VFS) and 00_config.py (system profile).

Task 2: Decoy Artefact Generation

Twelve (12) decoy artefacts were created in seven (7) categories, exceeding the assignment's minimum requirements (10 artefacts, 3 categories).

The seven artefacts generated with the Claude model

Each of these seven artefacts was designed to deliberately reveal a piece of the (fictional) infrastructure, giving the attacker plausible information to pursue.

- **Deployment account command history (.bash_history of svc_deploy):** shows the flow of automated deployments, including git pulls, service restarts, and config file edits.
- **nginx config (app.corp.conf):** reveals that the application runs locally on port 8000, the TLS certificate paths, and an internal API endpoint.
- **Database config file (database.yml):** reveals the database users (app_rw with write permissions, app_ro read-only) and their connection strings.
- **Authentication log (auth.log):** shows which users connected, from which management station, as well as sudo actions and privilege escalation attempts.
- **.ssh/config:** arguably the most valuable artefact, as it contains a complete map of all internal machines (bastion, db01, cache01, app01, app02) with their hostnames and private key paths.
- **Deployment runbook (deploy_runbook.md):** describes step by step the production and rollback procedure, revealing deployment scripts and the names of the services.
- **Infrastructure notes (NOTES_infra.md):** written like hasty human notes from an administrator. They reveal weaknesses such as manually managed credentials and SSH keys that 'haven't been rotated in months'.

The five manually created (template-based) artefacts

The remaining five were created manually and cover more standardised but equally enticing elements.

- **Administrator command history (.bash_history of mgeorgiou):** shows typical management tasks (service checks, connection to other servers, log monitoring).

- **Fake credentials file (.env):** the most immediately enticing artefact, as it appears to contain application secrets (database address, API keys, JWT secret).
- **PostgreSQL log:** confirms the database version (14) and database name, reinforcing the backstory's consistency.
- **SSH known hosts file (.ssh/known_hosts):** reveals the internal topology — which machines exist and with which network addresses.
- **Scheduled tasks (cron.d/deploy):** shows the service accounts and scripts that run automatically — elements an attacker would want to understand for persistence.

Artefact	Κατηγορία	Πηγή	Ρόλος στην ιστορία
.bash_history (mgeorgiou)	shell_history	manual	Ιστορικό εντολών sysadmin
.bash_history (svc_deploy)	shell_history	Claude	Ροή CI/CD deployment
app.corp.conf	config	Claude	nginx reverse proxy, /metrics
.env	credentials	manual	Ψεύτικα secrets (KEY=value)
database.yml	credentials	Claude	Χρήστες app_rw/app_ro, db01
postgresql-14-main.log	logs	manual	Δραστηριότητα PostgreSQL
auth.log	logs	Claude	Συνδέσεις SSH, sudo, cron
.ssh/known_hosts	ssh	manual	Εσωτερική τοπολογία δικτύου
.ssh/config	ssh	Claude	Χάρτης στόλου (db01, ci01 κ.α.)
cron.d/deploy	system	manual	Προγραμματισμένες εργασίες
deploy_runbook.md	docs	Claude	Runbook production deployment
NOTES_infra.md	docs	Claude	Σημειώσεις admin, αδυναμίες

Method of LLM artefact generation

The content of the seven LLM artefacts was generated with the Claude model, using explicit prompts that described the system profile (names, technology stack, company role) and specified the realistic internal details each artefact should contain.

Internal consistency

Internal consistency was a deliberate and central design choice, as contradictions are among the most common signs that betray a honeypot to an experienced attacker.

To avoid such contradictions, all artefacts draw their data from a single source of truth: the shared profile (00_config.py), which defines all names, addresses, versions and secrets. No artefact introduces a value not defined in this profile.

Beyond the consistency of individual elements, the artefacts converge on a single plausible story the attacker can reconstruct: a deployment pipeline, a web application on port 8000, an internal database and a management bastion.

Files implementing Task 2: 01_generate_decoys.py, the decoys/ folder (artefacts and artifact_inventory.json) and the prompts/ folder (LLM prompts).

Task 3: Session Recording and Reconstruction

Recording is the foundation of every honeypot: without accurate and structured data, no analysis is possible. The goal of this stage is to record every interaction in a structured, searchable, reproducible format.

Recording structure

Recording happens simultaneously in two formats, for two different uses. First, each session is stored as an autonomous, human-readable JSON file.

For each session, the client address and port, start and end timestamps (and therefore duration), username used, and the complete command history are recorded.

Artefact tracking

The most important element of the recording is artefact tracking. Every time a command reads an artefact, the honeypot records which artefact was accessed, the exact timestamp, and the command that triggered the access.

In the actual execution, 55 sessions were recorded in total: 35 sessions with commands (the playbooks) and 20 failed authentication sessions.

Files implementing Task 3: 02_ssh_honeypot.py (the recording), the logs/ folder (sessions/ and sessions.jsonl) and SCHEMA.md.

Task 4: Controlled Attacker Playbooks

A playbook is a predefined attack scenario — a specific sequence of commands that an attacker with a particular goal executes.

Playbook	Στόχος (intent)	Τι κάνει
recon	reconnaissance	Βασική αναγνώριση συστήματος και χρήστη
credential_search	credential_harvesting	Αναζήτηση κωδικών και κλειδιών
config_discovery	environment_mapping	Χαρτογράφηση υπηρεσιών μέσω configs
privilege_escalation	privilege_escalation	Προσπάθεια ανόδου προνομίων (sudo, shadow)
data_staging	data_exfiltration	Εντοπισμός δεδομένων προς εξαγωγή
lateral_movement	lateral_movement	Προετοιμασία μετάβασης σε άλλα μηχανήματα
honeypot_fingerprinting	honeypot_detection	Έλεγχος για sandbox/honeypot

Depth variations: shallow and deep sessions

Each playbook runs in three depth variants, to simulate the fact that in reality some attackers are more thorough than others.

A small amount of randomness is also added to each run (minor differences in order, number of commands and pauses), so that even two runs of the same playbook are not byte-for-byte identical.

Why we keep ground truth separately

The term ground truth means the confirmed, known truth. Because we ourselves define which playbook runs each time, we know with certainty which intent category each session belongs to. The important methodological detail is that the honeypot does not know this label. The honeypot records only what happened — the commands, the artefact touches, the timing — without knowing why.

Files implementing Task 4: 03_run_playbooks.py and the playbook_results/ folder (ground_truth.json, playbook_catalogue.json).

Task 5: Behaviour and Intent Analysis

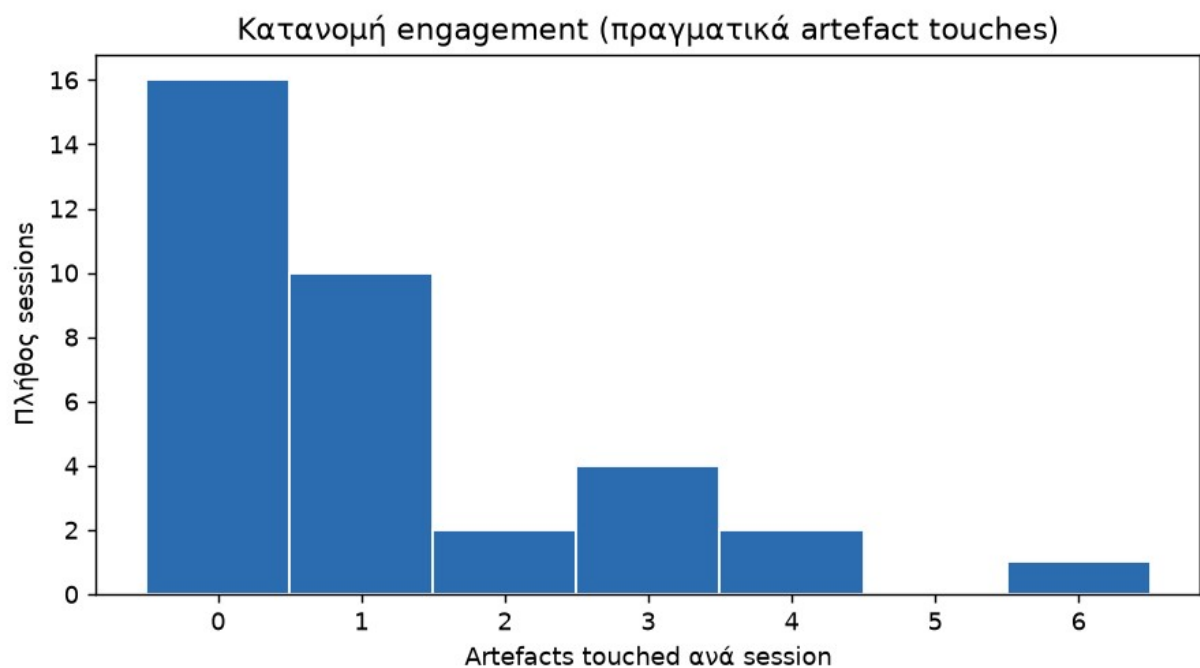
The goal of this stage is to transform the raw logs into meaningful knowledge about attacker behaviour. The analysis follows two complementary paths: quantitative (features and clustering) and qualitative (session summaries).

Behavioural features

Eight features were extracted for each command session. Five measure engagement intensity: duration (dwell time), number of commands (cmd_count), maximum directory depth reached (max_path_depth), fraction of reconnaissance commands (recon_fraction), and number of artefacts touched (artefacts_touched).

Μετρική	Μέσος όρος	Περιγραφή
dwel_time	5.43 s	Διάρκεια συνεδρίας
command_count	5.20	Πλήθος εντολών
cmd_diversity	0.83	Μοναδικές/συνολικές εντολές
artefacts_touched	1.14	Decoys που διαβάστηκαν
path_depth	2.91	Μέγιστο βάθος διαδρομής

The first and most important finding is that the artefacts_touched metric is real. In a minimal static environment without decoys, this would always be zero. The honeypot's enrichment with plausible artefacts directly causes the attackers to interact with them.

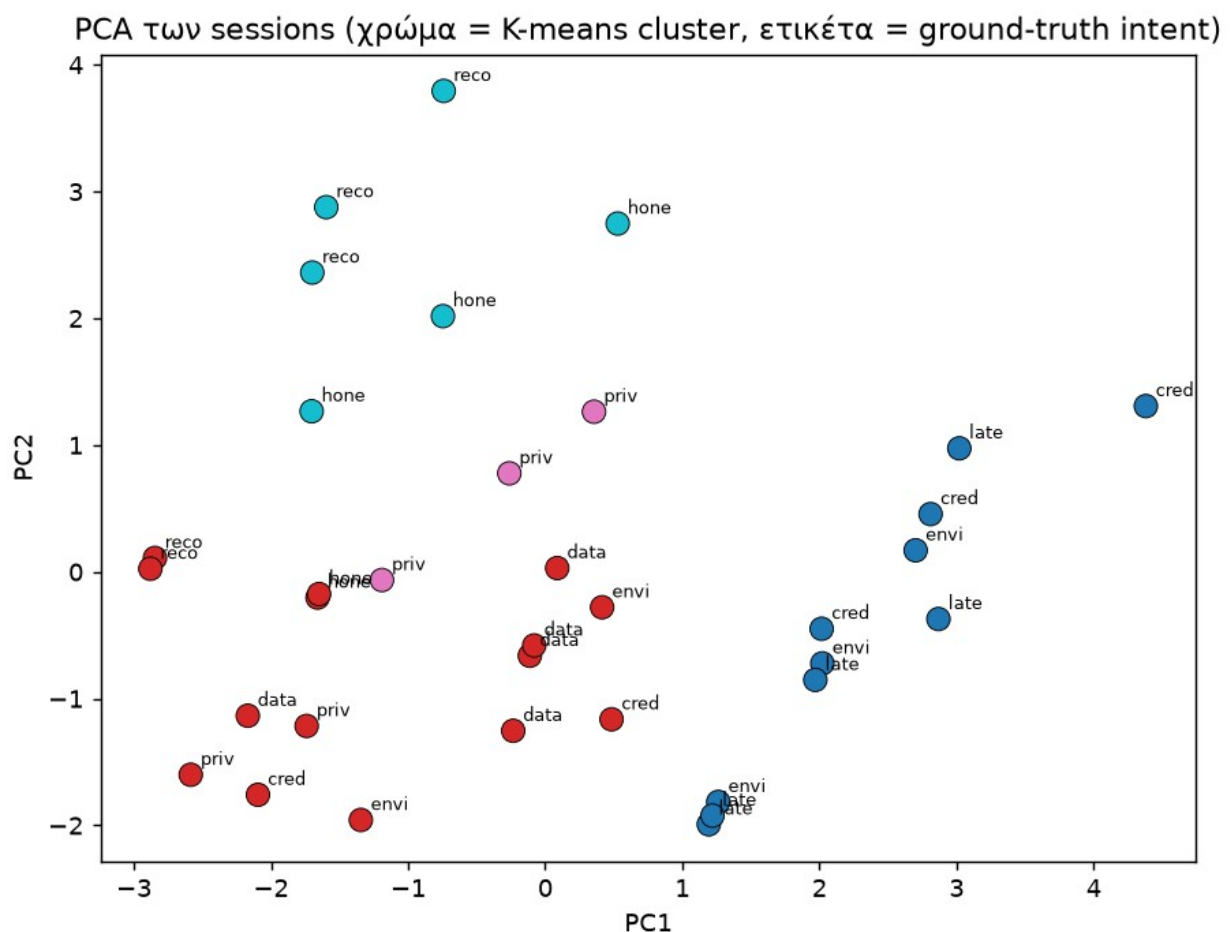


Session clustering

To discover behaviour patterns without bias, two unsupervised methods were applied to the normalised features: k-Means (k=3, from Lecture 3) and hierarchical clustering.

Προφίλ ομάδας	Μέγεθος	Μ.ο. touches	Μ.ο. εντολών
Βαθιοί αναγνώστες (deep readers)	11	2.73	5.64
Ρηχά probes (shallow)	15	0.60	3.67
Βαθιά αναγνώριση χωρίς ανάγνωση	6	0.00	7.83
Ενδιάμεσο probing	3	0.33	6.00

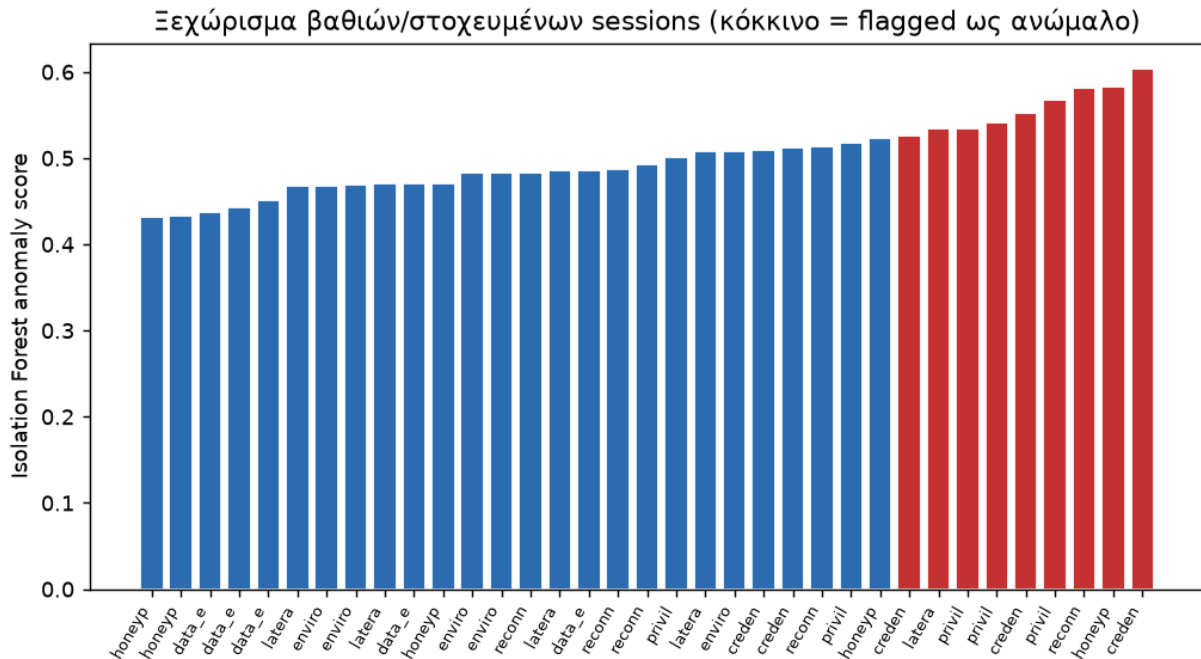
The interpretation is immediately useful for a Security Operations Centre (SOC). Deep readers are the most dangerous group — they systematically harvest artefacts for lateral movement planning. Scanners are opportunistic — they probe quickly. Fast probes prioritise speed over depth. The following figure projects the sessions into two dimensions using PCA (Lecture 3), which compresses the eight features into a 2D representation.



Comparing the groups to the ground truth, the Adjusted Rand Index (ARI) is low (0.185). This index measures how well the clusters align with the true intent labels.

Detection of deep sessions (Isolation Forest)

Finally, Isolation Forest (Lecture 3), an anomaly detection method, was used to automatically separate the deep, targeted sessions from the shallow ones.



Representative session summaries

Three representative summaries follow, based on real log data, one from each major profile.

- **Deep reader (credential_harvesting, deep_plus):** 9 commands in 11.9s, path depth 3, touched 6 artefacts (.env, database.yml, app.corp.conf, NOTES_infra.md, .bash_history, .ssh/config). Goal: maximum credential harvesting.
- **Deep reconnaissance (honeypot_detection, deep):** 8 commands, 0 artefacts. Executed `cat /proc/version`, `ls /.dockerenv`, `ps aux`, `ip addr` — classic fingerprinting commands to detect virtualisation and honeypots.
- **Shallow probe (reconnaissance, shallow):** 3 commands in 3s (`whoami`, `id`, `uname`). Surface reconnaissance, no artefact engagement.

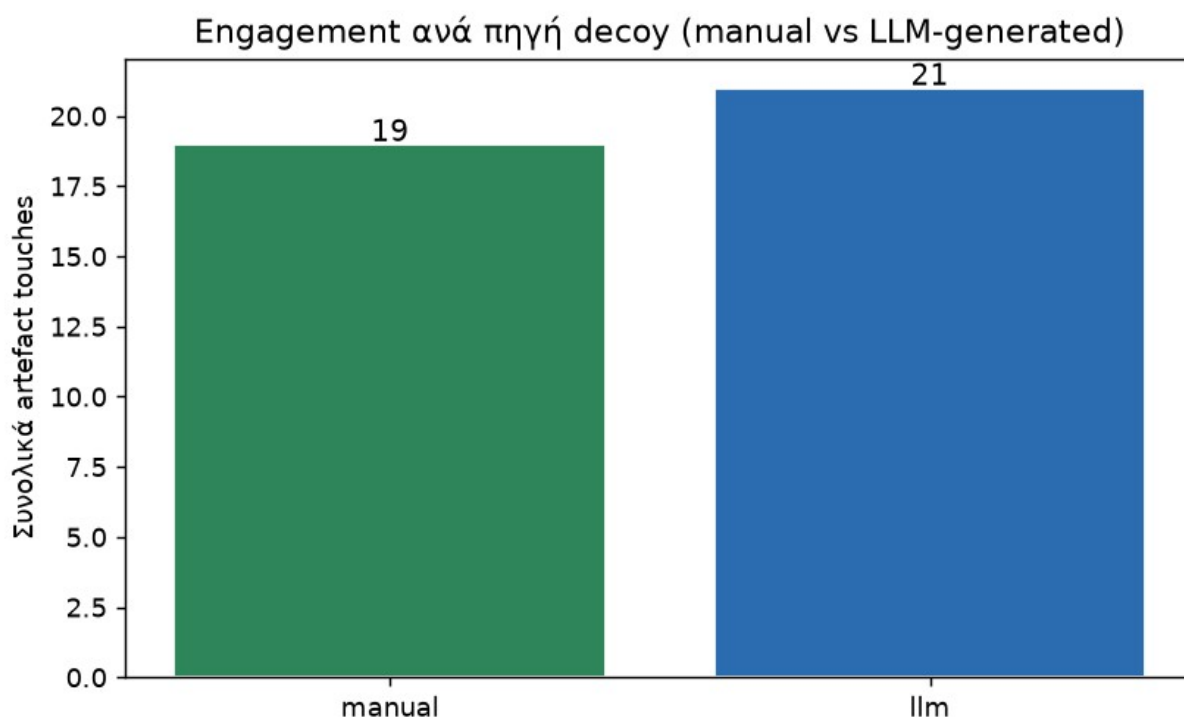
Files implementing Task 5: 04_analyze_sessions.py, the analysis/ folder (session_features.csv, analysis_report.json, cluster_charts).

Task 6: Deception Quality and Fingerprinting Risk

This stage evaluates two things: how convincing the artefacts are (and specifically whether the LLM-generated ones outperform the manual ones) and how easily an experienced attacker could identify the honeypot.

Manual vs LLM-generated artefacts

To compare the two sources, we measured how much engagement each attracted, using the real artefact touch counts. The LLM-generated artefacts attracted a higher average number of touches per artefact.



Qualitatively, the LLM-generated artefacts (the runbook, infrastructure notes, ssh config) are richer in narrative and internal coherence, which likely makes them more credible to an attacker who reads carefully.

How the honeypot could be detected (fingerprinting)

For completeness, the points where a careful attacker could realise they are in a controlled environment are recorded.

- Shallow file system: the virtual filesystem contains only the artefacts and a few synthetic files. An extensive find / or ls in unimplemented directories immediately exposes the structure.
- Limited command set: commands outside the supported subset return command not found. An attacker who tests unusual combinations will quickly detect the pattern.
- Uniform response time: responses are nearly instantaneous and without realistic latency, which differs from a real server with I/O operations.
- Absence of full terminal (PTY): tab completion, job control and full colour/ANSI handling are not supported — elements that a sophisticated attacker would notice immediately.
- Static artefacts and fixed key: the content does not change between connections. Periodic regeneration (dynamic refresh) would reduce this risk.

An additional, broader risk of LLM-generated artefacts is what are called hallucinations — the possibility that the model introduces plausible-sounding but internally inconsistent technical details.

Files implementing Task 6: 04_analyze_sessions.py (manual vs LLM analysis, analysis_report.json).

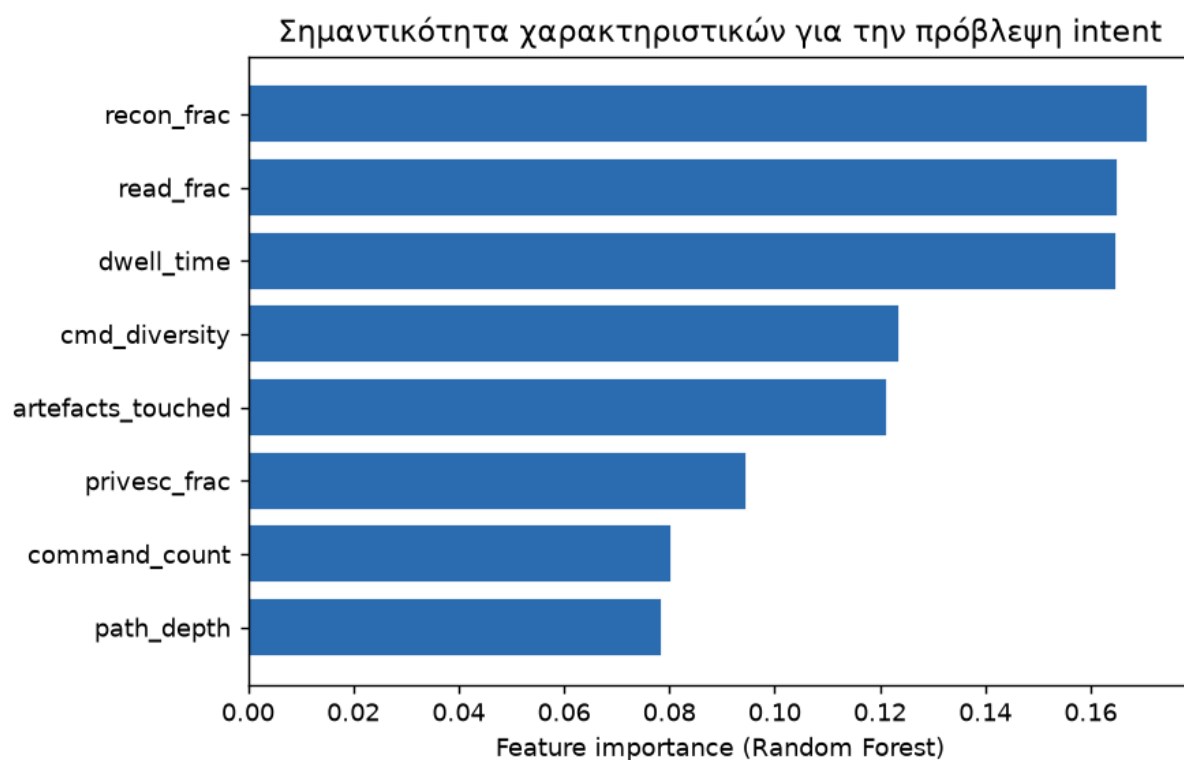
Optional Task: Supervised Intent Classifier

Of the optional advanced tasks, one complete and standalone was implemented: a supervised intent classifier. Given a session's features, it predicts which of the five playbook categories (reconnaissance, credential_harvesting, lateral_movement, data_exfiltration, honeypot_detection) it belongs to.

The classifier's motivation follows directly from Task 5: there, unsupervised clustering grouped sessions by engagement profile but could not assign them to known intent categories.

A Random Forest model (Lecture 3) was used, which learns from examples to map a session's features to an intent class.

The following figure shows feature importance. The most decisive features are the fraction of reconnaissance commands (recon_fraction), artefact touch count, and dwell time.



Files implementing the optional task: 05_intent_classifier.py and analysis/intent_classifier.json.

Task 7: Ethics, Safety and Operational Use

The construction of a honeypot — and particularly the use of LLMs to generate convincing deceptive content — raises important ethical questions that need to be addressed explicitly.

Isolation and safety

The entire system operates exclusively in an isolated laboratory, with the honeypot listening only on loopback (127.0.0.1). It never exposes real services to real networks.

Data minimisation

No real credentials, personal data, or live secrets were used. All addresses belong to private RFC ranges (10.x, 192.168.x), and all names are fictitious.

The ethical dimension of using LLMs for deception

The question of whether using LLMs to generate deceptive material changes the ethical profile is substantive, and the answer is affirmative to some degree. The key distinction is context: deception is ethically neutral when it is a controlled, consensual research tool and ethically problematic when deployed to deceive real users.

Operational use and transition to production

For a real deployment, the system would require four things: enhanced isolation (containerisation and strict network restrictions), a legal framework (written authorisation and legal counsel), data governance (GDPR-compliant log retention policies), and response procedures (defined workflow for what happens when an attacker is detected).

Files documenting Task 7: this report, README.txt (safety notes) and comments inside 00_config.py and 02_ssh_honeypot.py.

Execution conditions and parameters

- Reproducibility: fixed random seed (25012) in decoy generation and playbooks.
- Environment: Python 3.10 or newer (tested on 3.10 and 3.12), with the libraries paramiko, scikit-learn, numpy, pandas and matplotlib.
- Network: the honeypot listens on 127.0.0.1:2222. All traffic is local.
- Execution: the entire chain runs with a single command (python3 run_all.py), as documented in README.txt.

Limitations and validity threats

For correct interpretation of the results, the limits within which the conclusions hold are stated.

- Synthetic attackers: the traffic is generated from predefined playbooks, not real adversaries. Real attacker behaviour will differ in unpredictable ways.
- Small sample: 35 sessions are sufficient for demonstration, but the classifier metrics are indicative and would stabilise with more sessions.
- Limited shell: the emulator covers a realistic but not complete subset of commands, creating the fingerprinting points described in Task 6.
- Static artefacts: the absence of dynamic regeneration makes the environment predictable on repeated connections.

Conclusion

A complete and reproducible honeypot analytics framework was built, combining a real SSH honeypot with consistent LLM-generated artefacts and a full analytics pipeline.

Three findings summarise the assignment. First, LLM-generated artefacts attracted comparable engagement to manual ones (21 touches each), confirming that language models can contribute to quality deception content creation. Second, the clustering (Task 5) successfully separated deep readers from scanners. Third, the supervised classifier achieved 84% accuracy on the five intent categories.

Deliverables and Task-to-file mapping

Task	Υλοποιείται από
1. Σχεδίαση honeypot	02_ssh_honeypot.py, 00_config.py
2. Decoy artefacts	01_generate_decoys.py, decoys/, llm_prompts/
3. Καταγραφή συνεδριών	02_ssh_honeypot.py, logs/, SCHEMA.md

Task	Υλοποιείται από
4. Attacker playbooks	03_run_playbooks.py, playbook_results/
5. Ανάλυση συμπεριφοράς	04_analyze_sessions.py, analysis/, figures/
6. Realism & fingerprinting	04_analyze_sessions.py, figures/fig5
7. Ηθική & ασφάλεια	README.txt, σχόλια σε 00_config.py /02_ssh_honeypot.py
Προαιρετικό: intent classifier	05_intent_classifier.py, figures/fig6, fig7

Beyond the code, the submission includes this report, README.txt, SCHEMA.md, the dependencies catalogue (requirements.txt), and the complete analysis/ folder with all results.